基础数论

冯东宇

清华大学

1 整除

2 质数

3 模算术

4 积性函数

5 例题

整除

设 n 为非负整数，d 为正整数，若 $\frac{n}{d}$ 为整数，则称 d 整除 n，记为 $d \mid n$。此时，称 d 为 n 的约数，或因数，或因子，而称 n 为 d 的倍数。

整除

设 n 为非负整数，d 为正整数，若 $\frac{n}{d}$ 为整数，则称 d 整除 n，记为 $d \mid n$。此时，称 d 为 n 的约数，或因数，或因子，而称 n 为 d 的倍数。

任何正整数都整除 0。

整除

设 n 为非负整数，d 为正整数，若 $\frac{n}{d}$ 为整数，则称 d 整除 n，记为 $d \mid n$。此时，称 d 为 n 的约数，或因数，或因子，而称 n 为 d 的倍数。

任何正整数都整除 0。

d 不整除 n 记为 $d \nmid n$。

最大公约数

设 a,b 为非负整数，d 为正整数，若 $d \mid a$ 且 $d \mid b$，则称 d 为 a 和 b 的公约数，或公因数，或公因子。

最大公约数

设 a, b 为非负整数，d 为正整数，若 $d \mid a$ 且 $d \mid b$，则称 d 为 a 和 b 的公约数，或公因数，或公因子。

a 和 b 的所有公约数中最大的数称为 a 和 b 的最大公约数，或最大公因数，或最大公因子，记为 $\gcd(a, b)$，有时简记为 (a, b)。

最大公约数

设 a, b 为非负整数，d 为正整数，若 $d \mid a$ 且 $d \mid b$，则称 d 为 a 和 b 的公约数，或公因数，或公因子。

a 和 b 的所有公约数中最大的数称为 a 和 b 的最大公约数，或最大公因数，或最大公因子，记为 $\gcd(a, b)$，有时简记为 (a, b)。

根据定义，显然有 $\gcd(a, b) = \gcd(b, a)$。

最大公约数

设 a, b 为非负整数，d 为正整数，若 $d \mid a$ 且 $d \mid b$，则称 d 为 a 和 b 的公约数，或公因数，或公因子。

a 和 b 的所有公约数中最大的数称为 a 和 b 的最大公约数，或最大公因数，或最大公因子，记为 $\gcd(a, b)$，有时简记为 (a, b)。

根据定义，显然有 $\gcd(a, b) = \gcd(b, a)$。

若 $\gcd(a, b) = 1$，则称 a 和 b 互质或互素。

最大公约数

由于任何正整数都是 0 和 0 的公约数，故 $\gcd(0,0)$ 不存在。

最大公约数

由于任何正整数都是 0 和 0 的公约数，故 $\gcd(0,0)$ 不存在。

对任意正整数 a，有 $\gcd(0,a)=a$。

最小公倍数

设 a,b 为正整数，m 为非负整数，若 $a \mid m$ 且 $b \mid m$，则称 m 为 a 和 b 的公倍数。

最小公倍数

设 a, b 为正整数，m 为非负整数，若 $a \mid m$ 且 $b \mid m$，则称 m 为 a 和 b 的公倍数。

a 和 b 的所有公倍数中最小的正数称为 a 和 b 的最小公倍数，记为 $\operatorname{lcm}(a, b)$。

最小公倍数

设 a, b 为正整数，m 为非负整数，若 $a \mid m$ 且 $b \mid m$，则称 m 为 a 和 b 的公倍数。

a 和 b 的所有公倍数中最小的正数称为 a 和 b 的最小公倍数，记为 $\operatorname{lcm}(a, b)$。

根据定义，显然有 $\operatorname{lcm}(a, b) = \operatorname{lcm}(b, a)$。

最小公倍数

一个显然而重要的公式是

$$\operatorname{lcm}(a,b) = \frac{ab}{\gcd(a,b)}$$

这个公式给出了 lcm 与 gcd 的关系。

最小公倍数

一个显然而重要的公式是

$$\operatorname{lcm}(a,b) = \frac{ab}{\gcd(a,b)}$$

这个公式给出了 lcm 与 gcd 的关系。

需要注意的是，在实际的代码实现中，为避免中间结果的溢出，应该使用 `a/gcd(a,b)*b` 而不是 `a*b/gcd(a,b)`。

gcd 的性质

设 a, b 为正整数且 $a > b$，有性质

$$\gcd(b, a) = \gcd(b, a - b)$$

由此可进一步得到性质

$$\gcd(b, a) = \gcd(b, a \bmod b)$$

gcd 的计算

```
int gcd(int a, int b){
    return b ? gcd(b, a%b) : a;
}
```

复杂度为 $O\big(\log \max(a,b)\big)$。

质数

设 $n \geq 2$ 为整数，若所有满足 $1 < k < n$ 的整数 k 都不是 n 的约数，则称 n 为质数或素数，否则称 n 为合数。

质数

设 $n \geq 2$ 为整数，若所有满足 $1 < k < n$ 的整数 k 都不是 n 的约数，则称 n 为质数或素数，否则称 n 为合数。

1 既不是质数，也不是合数。

质数个数

定义 $\pi(n)$ 为不大于 n 的质数的个数，可以证明

$$\pi(n) = O(\frac{n}{\log n})$$

唯一分解定理

设 $n \geq 2$ 为整数，则有唯一的分解式

$$n = \prod_{i=1}^{m} {p_i}^{k_i}$$

其中 $p_1 < p_2 < \cdots < p_m$ 为质数，k_i 为正整数。

唯一分解定理

设 $n \geq 2$ 为整数，则有唯一的分解式

$$n = \prod_{i=1}^{m} {p_i}^{k_i}$$

其中 $p_1 < p_2 < \cdots < p_m$ 为质数，k_i 为正整数。

可以证明 $m = O(\log \log n)$。

分解方法一

```
vector<int> factor(int n){
    vector<int> f;
    for(int i = 2; i <= n; ++i)
        while(n%i == 0){
            f.push_back(i);
            n /= i;
        }
    return f;
}
```

分解方法二

```
vector<int> factor(int n){
    vector<int> f;
    for(int i = 2; i*i <= n; ++i)
        while(n%i == 0){
            f.push_back(i);
            n /= i;
        }
    if(n > 1)
        f.push_back(n);
    return f;
}
```

分解方法三

```
vector<int> p;
vector<int> factor(int n){
    vector<int> f;
    for(int i = 0; i < p.size(); ++i){
        if(p[i]*p[i] > n)
            break;
        while(n%p[i] == 0){
            f.push_back(p[i]);
            n /= p[i];
        }
    }
    if(n > 1)
        f.push_back(n);
    return f;
}
```

复杂度分析

- 方法一的复杂度为 $O(n)$。
- 方法二的复杂度为 $O(\sqrt{n})$。
- 方法三的复杂度为 $O(\frac{\sqrt{n}}{\log n})$。

求所有不大于 n 的质数

容易想到对所有不大于 n 的正整数逐个判断是不是质数，但这不是个好主意。

调和级数

设调和级数前 n 项的和为 $f(n)$，即

$$f(n) = \sum_{i=1}^{n} \frac{1}{i}$$

可以证明

$$f(n) = O(\log n)$$

筛法一

从 2 到 n 枚举整数 i，标记大于 i 且不大于 n 的 i 的倍数。枚举到 i 时，若 i 没有被标记过，则 i 为质数。

筛法一

从 2 到 n 枚举整数 i，标记大于 i 且不大于 n 的 i 的倍数。枚举到 i 时，若 i 没有被标记过，则 i 为质数。

复杂度为

$$O\left(\sum_{i=1}^{n}\frac{n}{i}\right)=O(n\log n)$$

筛法一

```
bool vis[N+1];
vector<int> p;
void sieve(){
    for(int i = 2; i <= N; ++i){
        if(!vis[i])
            p.push_back(i);
        for(int j = i*2; j <= N; j += i)
            vis[j] = 1;
    }
}
```

筛法二

从 2 到 n 枚举整数 i，若 i 是质数，标记大于 i 且不大于 n 的 i 的倍数。枚举到 i 时，若 i 没有被标记过，则 i 为质数。

筛法二

从 2 到 n 枚举整数 i，若 i 是质数，标记大于 i 且不大于 n 的 i 的倍数。枚举到 i 时，若 i 没有被标记过，则 i 为质数。

可以证明复杂度为 $O(n \log \log n)$。这种筛法有个不好记的名字。

筛法二

```
bool vis[N+1];
vector<int> p;
void sieve(){
    for(int i = 2; i <= N; ++i)
        if(!vis[i]){
            p.push_back(i);
            for(int j = i*2; j <= N; j += i)
                vis[j] = 1;
        }
}
```

筛法三

从 2 到 n 枚举整数 i，再从小到大枚举所有不大于 i 的最小质因子的质数 p_0，标记 ip_0。显然，枚举到的 p_0 总是 ip_0 的最小质因子，而更大的 p_0 均不可能是 ip_0 的最小质因子。同样，枚举到 i 时，若 i 没有被标记过，则 i 为质数。

筛法三

从 2 到 n 枚举整数 i，再从小到大枚举所有不大于 i 的最小质因子的质数 p_0，标记 ip_0。显然，枚举到的 p_0 总是 ip_0 的最小质因子，而更大的 p_0 均不可能是 ip_0 的最小质因子。同样，枚举到 i 时，若 i 没有被标记过，则 i 为质数。

每个合数都只会在其最小的质因子被枚举到时被标记，故复杂度为 $O(n)$。这种筛法称为欧拉筛或线性筛。

筛法三

```
bool vis[N+1];
vector<int> p;
void sieve(){
    for(int i = 2; i <= N; ++i){
        if(!vis[i])
            p.push_back(i);
        for(int j = 0; i*p[j] <= N; ++j){
            vis[i*p[j]] = 1;
            if(i%p[j] == 0)
                break;
        }
    }
}
```

取模

设 a 为整数，n 为正整数，定义

$$a \bmod n = \begin{cases} a - \left\lfloor \frac{a}{n} \right\rfloor n & a \geq 0 \\ -(-a \bmod n) & a < 0 \end{cases}$$

这与 C++ 中的取模运算符的行为一致。

同余

若 $(a-b) \bmod n = 0$，则称 a 和 b 同余，记为

$$a \equiv b \pmod{n}$$

a 和 b 均为正数时，此式等价于 $a \bmod n = b \bmod n$。

模算术的基本性质

设 $a_0 = a \bmod n$, $b_0 = b \bmod n$，有

$$\begin{aligned} a + b &\equiv a_0 + b_0 \pmod{n} \\ ab &\equiv a_0 b_0 \pmod{n} \end{aligned}$$

模算术的基本性质

对任意正整数 k，有

$$a \bmod n = (a \bmod kn) \bmod n$$

若 $k \mid a$，有

$$\frac{a}{k} \bmod n = \frac{a \bmod kn}{k}$$

取模优化

```
template<class T>
inline T mod(T a){
    return a<p ? a : a-p;
}
```

用内嵌汇编实现模乘法

```
u32 mul(u32 a, u32 b){
    asm(
        "mul %%ebx\n"
        "div %%ecx"
        : "=d"(a)
        : "a"(a), "b"(b), "c"(p)
    );
    return a;
}
```

快速幂

```
int pow(int a, int n){
    int s = 1;
    for(; n; n >>= 1){
        if(n&1)
            s = (ll)s*a%p;
        if(n>1)
            a = (ll)a*a%p;
    }
    return s;
}
```

快速乘

```
u64 mul(u64 a, u64 b){
    u64 s = 0;
    for(; b; b >>= 1){
        if(b&1)
            s = mod(s+a);
        if(b>1)
            a = mod(a+a);
    }
    return s;
}
```

注记

快速幂有一个更直观的名字：平方求幂法。

类比这个名字，快速乘可以称为“加倍求积法”。毕竟，快速乘不仅完全不快，反而非常慢，称其为“快速”乘显得有些荒唐。

用内嵌汇编实现 64 位模乘法

```
u64 mul(u64 a, u64 b){
    asm(
        "mulq %%rbx\n"
        "divq %%rcx"
        : "=d"(a)
        : "a"(a), "b"(b), "c"(p)
    );
    return a;
}
```

用 long double 实现 64 位模乘法

```
u64 mul(u64 a, u64 b, u64 n){
    u64 q = (long double)a*b/n;
    return (a*b-q*n+n)%n;
}
```

只要编译器支持 long double 且 $0 \le a, b < n \le 2^{63}$，理论上不会出错。

模乘法的逆元

设 a 为整数，n 为正整数，若整数 b 满足

$$ab \equiv 1 \pmod{n}$$

则称 b 为 a 模 n 的逆元。

模乘法的逆元

容易证明以下结论：

- 当且仅当 $\gcd(a, n) = 1$ 时，a 模 n 的逆元存在。
- 如果 b_1, b_2 为 a 模 n 的逆元，则必有 $b_1 \equiv b_2 \pmod{n}$，即 a 模 n 的逆元在模 n 意义下唯一。

模乘法的逆元

容易证明以下结论：

- 当且仅当 $\gcd(a, n) = 1$ 时，a 模 n 的逆元存在。
- 如果 b_1, b_2 为 a 模 n 的逆元，则必有 $b_1 \equiv b_2 \pmod{n}$，即 a 模 n 的逆元在模 n 意义下唯一。

由于 a 的逆元唯一，在同余式中可以记为 a^{-1} 或 $\frac{1}{a}$。可以定义 $\frac{1}{a} \bmod n$ 为 a 模 n 的逆元中绝对值最小的数，并取与 a 相同的符号。

费马小定理

设 p 为质数，a 为与 p 互质的整数，则有

$$a^{p-1} \equiv 1 \pmod{p}$$

费马小定理

设 p 为质数，a 为与 p 互质的整数，则有

$$a^{p-1} \equiv 1 \pmod{p}$$

由此可知

$$a \cdot a^{p-2} \equiv 1$$
$$a^{-1} \equiv a^{p-2} \pmod{p}$$

从而可以用快速幂计算 a 模 p 的逆元。

预处理逆元

设 p 为质数，n 为小于 p 的正整数，求 $[1,n]$ 中的每个整数模 p 的逆元。

预处理逆元

设 p 为质数，n 为小于 p 的正整数，求 $[1, n]$ 中的每个整数模 p 的逆元。

若对每个数分别求逆元，复杂度为 $O(n \log p)$。

预处理逆元

设 i 为 $[1,n]$ 中的整数，有

$$p=\left\lfloor\frac{p}{i}\right\rfloor i+(p \bmod i)$$

从而有

$$\frac{1}{i}\equiv-\frac{\left\lfloor\frac{p}{i}\right\rfloor}{p \bmod i} \pmod p$$

由于 $p \bmod i<i$，可以 $O(n)$ 递推计算。

预处理逆元

注意到 $\frac{1}{i!}=\frac{i+1}{(i+1)!}$，故只需要 $O(\log p)$ 算出 $\frac{1}{n!}$，之后就可以 $O(n)$ 递推计算所有 $\frac{1}{i!}$，从而算出 $\frac{1}{i}$。

欧拉 φ 函数

定义 $\varphi(n)$ 为不大于 n 的正整数中与 n 互质的数的个数，即

$$\varphi(n) = \sum_{i=1}^{n} [\gcd(i, n) = 1]$$

其中 $[x]$ 在 x 成立时为 1，否则为 0。

欧拉定理

设 $n \geq 2$ 为整数，a 为与 n 互质的整数，则有

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

特别地，若 n 为质数，有 $\varphi(n) = n - 1$，即得到费马小定理。

欧拉定理

设 $n \geq 2$ 为整数，a 为与 n 互质的整数，则有

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

特别地，若 n 为质数，有 $\varphi(n) = n - 1$，即得到费马小定理。

由此可知

$$a^{-1} \equiv a^{\varphi(n)-1} \pmod{n}$$

欧拉定理

设 $n \geq 2$ 为整数，a 为与 n 互质的整数，则有

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

特别地，若 n 为质数，有 $\varphi(n) = n - 1$，即得到费马小定理。

由此可知

$$a^{-1} \equiv a^{\varphi(n)-1} \pmod{n}$$

由于计算 $\varphi(n)$ 的复杂度较高，一般不用这个公式计算 a 模 n 的逆元。

降幂公式

设 n, k 为正整数，a 为任意整数，总有

$$a^k \equiv a^{\min(k,k_0)} \pmod{n}$$

其中 $k_0 = \left(k \bmod \varphi(n)\right) + \varphi(n)$。

欧几里得算法

设 a, b 为正整数且 $a > b$，则存在正整数数组 q, r，满足

$$\begin{aligned} a &= q_0 b + r_0 \\ b &= q_1 r_0 + r_1 \\ r_0 &= q_2 r_1 + r_2 \\ &\vdots \\ r_{n-2} &= q_n r_{n-1} + r_n \\ r_{n-1} &= q_{n+1} r_n \end{aligned}$$

其中 $b > r_0 > r_1 > \cdots > r_n$。

欧几里得算法

可以注意到

$$\gcd(a,b) = \gcd(b,r_0) = \cdots = \gcd(r_{n-1},r_n) = r_n$$

此式给出了前述的求 $\gcd(a,b)$ 的过程。

扩展欧几里得算法

通过不断地应用公式

$$r_k = r_{k-2} - q_k r_{k-1}$$

可以求出可能为负数的 x, y，使得 $r_n = ax + by$。

扩展欧几里得算法

通过不断地应用公式

$$r_k = r_{k-2} - q_k r_{k-1}$$

可以求出可能为负数的 x, y，使得 $r_n = ax + by$。

特别地，若 $\gcd(a, b) = 1$，则有 $ax \equiv 1 \pmod{b}$，即 x 为 a 模 b 的逆元。

扩展欧几里得算法

```
int gcd(int a, int b, int& x, int& y){
    if(b){
        int d = gcd(b, a%b, y, x);
        // d = y*b + x*(a%b)
        //   = y*b + x*(a-(a/b)*b)
        //   = y*b + x*a - x*(a/b)*b
        //   = x*a + (y-x*(a/b))*b
        y -= a/b*x;
        return d;
    }
    x = 1;
    y = 0;
    return a;
}
```

中国剩余定理

求解同余方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \quad \vdots \\ x \equiv a_n \pmod{m_n} \end{cases}$$

其中 $m_1, m_2, \ldots, m_n$ 两两互质。

中国剩余定理

设 $M = \prod_{i=1}^{n} m_i$，可以构造解

$$x = \sum_{i=1}^{n} M_i(a_i {M_i}^{-1} \bmod m_i)$$

其中 $M_i = \frac{M}{m_i}$。可以证明解在模 M 意义下唯一。

中国剩余定理

设 $M = \prod_{i=1}^{n} m_i$，可以构造解

$$x = \sum_{i=1}^{n} M_i(a_i {M_i}^{-1} \bmod m_i)$$

其中 $M_i = \frac{M}{m_i}$。可以证明解在模 M 意义下唯一。

上述内容称为中国剩余定理或 CRT。

扩展 CRT

求解同余方程组

$$x \equiv a_i \pmod{m_i}$$

不要求 m_i 两两互质。

扩展 CRT

将每个方程拆成若干个方程

$$x \equiv a_i \pmod{p_{i,j}^{k_{i,j}}}$$

其中 $m_i = \prod p_{i,j}^{k_{i,j}}$ 为 m_i 的分解式。对每个质数 p，合并对应的所有方程，从而转化为模数两两互质的情形。若合并过程中出现矛盾，则原方程组无解。

积性函数

设 $f(n)$ 为定义在正整数上的函数，如果 $f(1)=1$，且对于任意正整数 a,b，只要 a 和 b 互质，就有

$$f(ab)=f(a)f(b)$$

则称 $f(n)$ 为积性函数。

积性函数

设 $f(n)$ 为定义在正整数上的函数，如果 $f(1)=1$，且对于任意正整数 a,b，只要 a 和 b 互质，就有

$$f(ab)=f(a)f(b)$$

则称 $f(n)$ 为积性函数。

如果不要求 a 和 b 互质，则称 $f(n)$ 为完全积性函数。

模乘法逆元

设 p 为质数，$f(n)=\frac{1}{n}$，则 $f(n)$ 在模 p 意义下为积性函数。

欧拉 φ 函数

对于任意正整数 n，有

$$\varphi(n) = n \prod_{p|n} (1 - \frac{1}{p})$$

故 $\varphi(n)$ 为积性函数。

约数函数

定义 $\sigma_x(n)$ 为 n 的所有约数的 x 次方和，即

$$\sigma_x(n) = \sum_{d|n} d^x$$

特别地，$\sigma_0(n)$ 为 n 的约数个数，常记为 $d(n)$ 或 $\tau(n)$；$\sigma_1(n)$ 为 n 的约数和，常简记为 $\sigma(n)$。

约数函数

可以证明对于任意 x，$\sigma_x(n)$ 均为积性函数。

约数函数

可以证明对于任意 x，$\sigma_x(n)$ 均为积性函数。

作为一个例子，有

$$\tau(n) = \prod_{i=1}^{m}(k_i - 1)$$

其中 $n = \prod_{i=1}^{m} {p_i}^{k_i}$。可以看出 $\tau(n)$ 为积性函数。

约数函数

对于质数 p 和正整数 k，有

$$\sigma_x(p^k) = \sum_{i=0}^{k}(p^i)^x = \sum_{i=0}^{k}(p^x)^i$$

从而当 $x \neq 0$ 时，有

$$\sigma_x(p^k) = \frac{(p^x)^{k+1} - 1}{p^x - 1}$$

积性函数的计算

一般地，若有分解式 $n = \prod_{i=1}^{m} {p_i}^{k_i}$，则有

$$f(n) = \prod_{i=1}^{m} f({p_i}^{k_i})$$

积性函数的计算

一般地，若有分解式 $n = \prod_{i=1}^{m} {p_i}^{k_i}$，则有

$$f(n) = \prod_{i=1}^{m} f({p_i}^{k_i})$$

求 $f({p_i}^{k_i})$ 的方法依赖于具体的 f 的性质，没有一般性的方法。

在线性筛过程中计算积性函数

```
bool vis[N+1];
vector<int> p;
int c[N+1], f[N+1];
void sieve(){
    for(int i = 2; i <= N; ++i){
        if(!vis[i]){
            p.push_back(i);
            for(int j = i, k = 1;; ++k){
                c[j] = j;
                f[j] = cal(i, k);
                if(j > N/i)
                    break;
                j *= i;
            }
        }
        for(int j = 0; i*p[j] <= N; ++j){
            vis[i*p[j]] = 1;
            if(i%p[j] == 0){
                c[i*p[j]] = c[i]*p[j];
                f[i*p[j]] = f[i/c[i]]*f[c[i]*p[j]];
                break;
            }
            c[i*p[j]] = p[j];
            f[i*p[j]] = f[i]*f[p[j]];
        }
    }
}
```